

WireGuard VPN Implementation Report

Project: Hybrid SOC Infrastructure – Secure Site-to-Cloud VPN

Technology: WireGuard VPN

VPN Server: AWS EC2 (Ubuntu 22.04 LTS)

Purpose: Secure communication between distributed Linux and Windows virtual machines across multiple private networks.

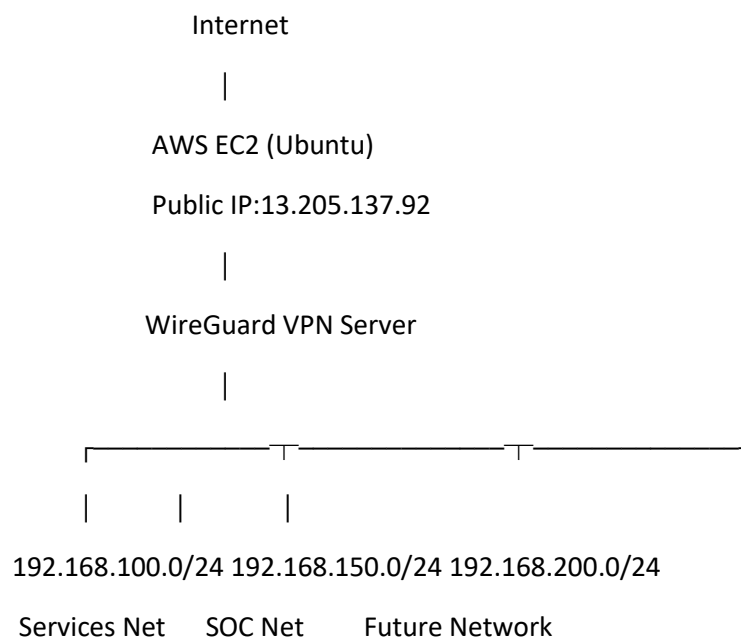
1. Objective

Implement a lightweight, secure Layer-3 VPN using **WireGuard** to interconnect all infrastructure components of the SOC lab over a single encrypted network.

The VPN was designed to:

- Secure communication between cloud and on-premises infrastructure.
- Eliminate direct exposure of internal services.
- Enable private access to SOC components.
- Allow communication between multiple isolated internal networks.
- Provide a centralized VPN gateway hosted on AWS.

2. Network Architecture



3. Infrastructure

VPN Server

Item	Value
Platform	AWS EC2
OS	Ubuntu 22.04 LTS
VPN Software	WireGuard
Interface	wg0
Transport	UDP
Listening Port	51820
Public Endpoint	13.205.137.92

VPN Clients

Implemented on

- Ubuntu Linux
- Windows Server 2022

Every client maintains its own key pair.

4. VPN Addressing Scheme

Server Addresses

The VPN server was assigned multiple addresses on a single WireGuard interface.

192.168.100.1/24

192.168.150.1/24

192.168.200.1/24

This enabled the server to function as the gateway for multiple logical VPN networks.

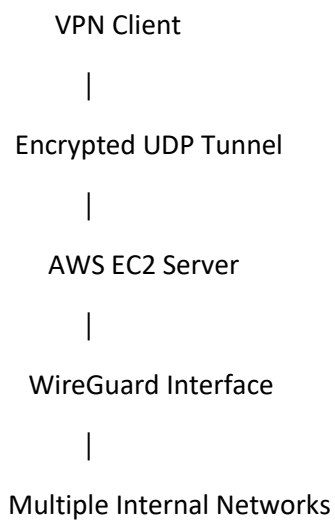
VPN Networks

Network	Purpose
192.168.100.0/24	Service Infrastructure
192.168.150.0/24	SOC Infrastructure
192.168.200.0/24	Reserved / Future Expansion

5. Machine Inventory

Host	VPN Address	Purpose
VPN Server	192.168.100.1 / 150.1 / 200.1	VPN Gateway
Web Server	192.168.100.10	Application Server
Web Server 2	192.168.100.50	Services
Honeypot	192.168.100.100	Threat Detection
Service VM	192.168.100.200	Reserved
MISP	192.168.150.10	Threat Intelligence
SOC Core	192.168.150.20	SIEM/Core Services
IDS	192.168.150.30	Intrusion Detection
AD-DS	192.168.150.40	Active Directory & DNS

6. WireGuard Architecture



7. Cryptography

WireGuard automatically provides

- Public-key authentication
- Private-key encryption
- Modern authenticated encryption
- Perfect Forward Secrecy (via Noise protocol framework)

Each peer possesses:

- Private Key
- Public Key

The server identifies every client exclusively by its public key.

8. WireGuard Server Configuration

Interface

[Interface]

Address = 192.168.100.1/24

Address = 192.168.150.1/24

Address = 192.168.200.1/24

ListenPort = 51820

PrivateKey = <Server Private Key>

DNS = 192.168.150.40

Peer Configuration

Each client was added as

[Peer]

PublicKey = <Client Public Key>

AllowedIPs = Client_IP/32

Example

AllowedIPs = 192.168.150.20/32

Each peer receives a unique VPN address.

9. Client Configuration

Each client contained

[Interface]

PrivateKey = Client Private Key

Address = Client_IP/24

DNS = 192.168.150.40

Peer

[Peer]

PublicKey = Server Public Key

Endpoint = 13.205.137.92:51820

AllowedIPs = 0.0.0.0/0

PersistentKeepalive = 25

After WSTunnel integration:

Endpoint = 127.0.0.1:51820

The WireGuard configuration otherwise remained unchanged.

10. Communication Flow

Client



WireGuard Encryption



UDP 51820



AWS Server



WireGuard



Private Network

After WSTunnel deployment:

Client



WireGuard



localhost



WSTunnel



HTTPS



AWS



WireGuard



VPN Network

11. Routing Strategy

WireGuard was configured as a full-tunnel VPN:

AllowedIPs = 0.0.0.0/0

Meaning

All outbound traffic is routed through the VPN unless overridden by a more specific route.

To preserve WSTunnel connectivity, a static host route to the AWS public IP was added via the physical network interface, preventing recursive routing.

12. Firewall Configuration

AWS Security Group

Allowed

Port	Purpose
22	SSH
80	HTTP
443	WSTunnel
UDP 51820 WireGuard	

nftables

Configured to permit

- SSH

- ICMP
 - WireGuard UDP
 - VPN forwarding
 - NAT masquerading
 - HTTPS (after WSTunnel deployment)
-

13. NAT Configuration

Source NAT

Outgoing Interface

↓

ens5

↓

Masquerade

Purpose

Enable VPN clients to access external networks while preserving return routing.

14. DNS Configuration

VPN clients use

192.168.150.40

Purpose

Active Directory integrated DNS server.

This enables

- Internal hostname resolution
 - Domain services
 - Centralized DNS management
-

15. Validation Commands

Interface

`sudo wg show`

Displays

- Interface status
 - Public keys
 - Peers
 - Handshake
 - Transfer statistics
-

Interface Creation

`sudo wg-quick up wg0`

Interface Shutdown

`sudo wg-quick down wg0`

Routing

`ip route`

Connectivity

`ping <VPN_IP>`

16. Monitoring

Successful tunnel establishment is indicated by

Latest Handshake

along with increasing

- Received bytes
- Sent bytes

A missing handshake indicates connectivity or configuration issues.

17. Design Decisions

Decision	Reason
AWS-hosted VPN server	Centralized secure access point
Multiple subnets on one interface	Simplified routing and scalability
Static peer addressing	Predictable management and ACLs
Public-key authentication	Eliminate shared secrets
Full-tunnel (AllowedIPs = 0.0.0.0/0)	Centralized traffic routing
DNS via AD-DS	Unified name resolution and domain integration

18. Advantages

- Lightweight VPN implementation
 - Low latency and high throughput
 - Minimal configuration overhead
 - Strong modern cryptography
 - Easy peer management
 - Supports Linux and Windows clients
 - Scalable to multiple private networks
 - Centralized VPN gateway for the SOC environment
-

19. Limitations Identified

- Native WireGuard relies on UDP (51820), which may be blocked by enterprise firewalls.
 - Full-tunnel configurations require careful route management to avoid recursive routing.
 - Public-key distribution must be managed securely.
 - WireGuard itself does not provide built-in user authentication or centralized identity management; peer authorization is based on configured public keys.
-

20. Key Findings

- A single WireGuard interface can host multiple IP subnets, allowing one VPN gateway to serve several logical networks.
- Assigning a unique /32 AllowedIPs entry for each peer ensures deterministic routing and avoids address conflicts.

- Using PersistentKeepalive = 25 helps maintain NAT mappings for clients behind NAT or restrictive firewalls.
- Integrating the VPN with the AD-DS DNS server (192.168.150.40) provides centralized name resolution for domain-joined and VPN-connected hosts.
- WireGuard served as the secure transport foundation for the broader SOC infrastructure, enabling private communication among MISP, SOC Core, IDS, AD-DS, web services, and other internal systems. The subsequent WSTunnel layer extended this design by allowing the same WireGuard deployment to operate over HTTPS where native UDP traffic was blocked.

This cannot reasonably fit into a single response while including all commands, explanations, troubleshooting, WSTunnel integration, DNS configuration, and routing fixes. I'll split it into **4 implementation phases**.

- **Phase 1:** WireGuard Server Configuration (AWS EC2)
- **Phase 2:** Linux Client Configuration + DNS + Routing + Validation
- **Phase 3:** WSTunnel Integration (Server & Client) + Service Configuration + Default Route Handling
- **Phase 4:** Windows Client Configuration + Troubleshooting + Verification + Best Practices

WireGuard VPN Configuration Guide

Phase 1 – WireGuard Server Configuration (AWS EC2)

1. Prerequisites

AWS EC2

Component	Value
OS	Ubuntu 22.04 LTS
Public IP	13.205.137.92
VPN Interface	wg0
VPN Port	51820/UDP

Security Group

Allow the following inbound rules:

Protocol	Port	Purpose
UDP	51820	WireGuard VPN

Protocol	Port	Purpose
TCP	22	SSH
TCP	443	WSTunnel (future)
TCP	80	Optional HTTP

2. Install WireGuard

Update packages

```
sudo apt update
```

```
sudo apt upgrade -y
```

Install WireGuard

```
sudo apt install wireguard -y
```

Verify

```
wg --version
```

3. Enable IP Forwarding

Temporary

```
sudo sysctl -w net.ipv4.ip_forward=1
```

Permanent

```
sudo nano /etc/sysctl.conf
```

Uncomment or add

```
net.ipv4.ip_forward=1
```

Apply

```
sudo sysctl -p
```

Verify

```
cat /proc/sys/net/ipv4/ip_forward
```

Expected

1

4. Generate Server Keys

Create WireGuard directory

```
sudo mkdir -p /etc/wireguard
```

```
cd /etc/wireguard
```

Generate keys

```
wg genkey | sudo tee server_private.key | wg pubkey | sudo tee server_public.key
```

Restrict permissions

```
sudo chmod 600 server_private.key
```

Display public key

```
cat server_public.key
```

Save this value—it will be required on every client.

5. Create Server Configuration

```
sudo nano /etc/wireguard/wg0.conf
```

Interface

[Interface]

Address = 192.168.100.1/24,192.168.150.1/24,192.168.200.1/24

ListenPort = 51820

PrivateKey = <SERVER_PRIVATE_KEY>

DNS = 192.168.150.40

Why multiple addresses?

The server serves as the gateway for multiple logical VPN networks:

Network	Purpose
192.168.100.0/24	Services
192.168.150.0/24	SOC Infrastructure
192.168.200.0/24	Reserved/Future

6. Add Clients

Generate a key pair on each client.

For every client, add one peer.

Example:

[Peer]

PublicKey = CLIENT_PUBLIC_KEY

AllowedIPs = 192.168.100.10/32

Additional examples:

MISP

[Peer]

PublicKey = <KEY>

AllowedIPs = 192.168.150.10/32

SOC Core

[Peer]

PublicKey = <KEY>

AllowedIPs = 192.168.150.20/32

IDS

[Peer]

PublicKey = <KEY>

AllowedIPs = 192.168.150.30/32

AD-DS

[Peer]

PublicKey = <KEY>

AllowedIPs = 192.168.150.40/32

Honeypot

[Peer]

PublicKey = <KEY>

AllowedIPs = 192.168.100.100/32

Important

Always assign **one unique /32 address** per client.

Never configure:

AllowedIPs = 192.168.100.0/24

for an individual peer, as it causes ambiguous routing.

7. Configure nftables

Your deployment used **nftables** instead of iptables.

Edit the ruleset:

```
sudo nano /etc/nftables.conf
```

Example configuration:

```
table inet filter {
```

```
    chain input {
```

```
        type filter hook input priority filter;
```

```
        policy drop;
```

```
        iif "lo" accept
```

```
        ct state established,related accept
```

```
        iifname "wg0" accept
```

```
        tcp dport 22 accept
```

```
        udp dport 51820 accept
```

```
ip protocol icmp accept  
}
```

```
chain forward {
```

```
type filter hook forward priority filter;
```

```
policy drop;
```

```
iifname "wg0" accept
```

```
ct state established,related accept  
}
```

```
chain output {
```

```
type filter hook output priority filter;
```

```
policy accept
```

```
}  
}
```

```
table ip nat {
```

```
chain postrouting {
```

```
type nat hook postrouting priority srcnat;
```

```
policy accept;
```



```
    oifname "ens5" masquerade
  }
}
```

Apply:

```
sudo systemctl enable nftables
```

```
sudo systemctl restart nftables
```

Verify:

```
sudo nft list ruleset
```

8. Start WireGuard

Enable at boot:

```
sudo systemctl enable wg-quick@wg0
```

Start:

```
sudo systemctl start wg-quick@wg0
```

Or manually:

```
sudo wg-quick up wg0
```

9. Verify Server

Check interface:

```
ip addr show wg0
```

Check status:

```
sudo wg show
```

Initially, peers will appear without a handshake until clients connect.

10. Useful Commands

Bring up:

```
sudo wg-quick up wg0
```

Bring down:

```
sudo wg-quick down wg0
```

Restart:

```
sudo systemctl restart wg-quick@wg0
```

Status:

```
sudo systemctl status wg-quick@wg0
```

Logs:

```
sudo journalctl -u wg-quick@wg0
```

11. Implementation Facts

- The VPN server was deployed on an AWS EC2 Ubuntu 22.04 instance.
- A single WireGuard interface (wg0) was assigned IPs from three VPN subnets (192.168.100.0/24, 192.168.150.0/24, and 192.168.200.0/24), allowing it to act as the gateway for multiple logical networks.
- Each peer was identified by its public key and assigned a unique /32 VPN address using AllowedIPs, ensuring deterministic routing.
- IP forwarding and NAT masquerading were enabled so VPN clients could communicate across networks and, if required, access external resources.
- The server firewall was implemented with **nftables**, permitting SSH, ICMP, WireGuard UDP traffic, forwarding over wg0, and source NAT on the outbound interface.
- This configuration established the foundation for all subsequent integrations, including WSTunnel over HTTPS, Active Directory DNS, ELK components, MISP, IDS, and Windows clients.

Next: Phase 2 – Linux Client Configuration, DNS Integration, Routing Configuration, and VPN Validation.

WireGuard VPN Configuration Guide

Phase 2 – Linux Client Configuration, DNS Integration, Routing & Validation

1. Client Prerequisites

Component	Value
OS	Ubuntu 22.04 LTS
Host	VM-SERVICES-01
VPN Interface	wg0
VPN Address	192.168.100.10
VPN Server	13.205.137.92

2. Install WireGuard

Update packages

```
sudo apt update
```

```
sudo apt upgrade -y
```

Install WireGuard

```
sudo apt install wireguard resolvconf -y
```

Verify

```
wg --version
```

3. Generate Client Keys

```
cd /etc/wireguard
```

```
wg genkey | sudo tee client_private.key | wg pubkey | sudo tee client_public.key
```

Secure private key

```
sudo chmod 600 client_private.key
```

Display public key

```
cat client_public.key
```

Copy this key to the WireGuard server.

4. Register Client on Server

Open the server configuration.

```
sudo nano /etc/wireguard/wg0.conf
```

Add a peer.

[Peer]

```
PublicKey = CLIENT_PUBLIC_KEY
```

```
AllowedIPs = 192.168.100.10/32
```

Restart server

```
sudo systemctl restart wg-quick@wg0
```

5. Configure Client

Create configuration.

```
sudo nano /etc/wireguard/wg0.conf
```

Contents

[Interface]

PrivateKey = CLIENT_PRIVATE_KEY

Address = 192.168.100.10/24

DNS = 192.168.150.40

[Peer]

PublicKey = SERVER_PUBLIC_KEY

Endpoint = 13.205.137.92:51820

AllowedIPs = 0.0.0.0/0

PersistentKeepalive = 25

6. Why These Parameters?

Address

Address = 192.168.100.10/24

VPN address assigned to this client.

DNS

DNS = 192.168.150.40

Uses the AD-DS server as the DNS resolver.

Benefits:

- Internal hostname resolution
 - Active Directory integration
 - Centralized DNS management
 - No need for public DNS inside the VPN
-

Endpoint

Endpoint = 13.205.137.92:51820

Public WireGuard server.

After WSTunnel implementation (Phase 3), this will change to:

Endpoint = 127.0.0.1:51820

AllowedIPs

AllowedIPs = 0.0.0.0/0

This creates a **full-tunnel VPN**.

Meaning:

All client traffic

|



WireGuard Tunnel

|



VPN Server

Advantages:

- Secure Internet access through the VPN
 - Centralized routing
 - Uniform security policy
-

PersistentKeepalive

PersistentKeepalive = 25

Maintains NAT mappings for clients behind routers or enterprise firewalls by sending a keepalive packet every 25 seconds.

7. Bring Up the Tunnel

Start

```
sudo wg-quick up wg0
```

Enable at boot

```
sudo systemctl enable wg-quick@wg0
```

8. Verify Connection

Interface

```
ip addr show wg0
```

Status

```
sudo wg show
```

Expected output

latest handshake

transfer

received

sent

If the handshake is absent, investigate:

- Server running
 - Correct keys
 - Port 51820 open
 - Firewall rules
 - Public IP
 - Client configuration
-

9. Connectivity Tests

Ping VPN gateway

ping 192.168.100.1

Ping another VPN client

ping 192.168.150.20

Check routes

ip route

View interface

ip -br address

10. Routing Behaviour

With:

AllowedIPs = 0.0.0.0/0

WireGuard automatically installs routes such as:

Default

↓

wg0

This means almost all traffic leaves through the VPN.

11. Cross-Network Communication

Your VPN server was configured with three subnets:

192.168.100.0/24

192.168.150.0/24

192.168.200.0/24

Combined with:

- IP forwarding enabled
- nftables forwarding rules
- NAT masquerading

Clients on different VPN subnets could communicate securely through the AWS gateway.

Example:

192.168.100.10

|



AWS VPN Gateway

|



192.168.150.40

This allowed components such as the web server, MISP, IDS, SOC Core, and AD-DS to communicate despite being on separate logical VPN networks.

12. DNS Verification

Check the configured resolver:

```
cat /etc/resolv.conf
```

Or:

```
resolvectl status
```

Expected DNS:

192.168.150.40

Test name resolution:

```
nslookup <internal-hostname>
```

or

```
dig <internal-hostname>
```

13. Useful Commands

Show peers

```
sudo wg show
```

Restart

```
sudo systemctl restart wg-quick@wg0
```

Stop

```
sudo wg-quick down wg0
```


Start

```
sudo wg-quick up wg0
```

Logs

```
sudo journalctl -u wg-quick@wg0
```

Routes

```
ip route
```

14. Validation Checklist

Check	Command	Expected
Interface exists	<code>ip addr show wg0</code>	wg0 present
VPN status	<code>sudo wg show</code>	Peer listed
Handshake	<code>sudo wg show</code>	Recent handshake
Routing	<code>ip route</code>	VPN routes installed
Gateway reachability	<code>ping 192.168.100.1</code>	Success
Cross-subnet reachability	<code>ping <VPN IP></code>	Success
DNS	<code>resolvectl status</code>	AD-DS DNS (192.168.150.40)

15. Important Findings

- Each Linux client was configured with a unique static VPN IP address and authenticated using its own public/private key pair.
 - The client used the **AD-DS server (192.168.150.40)** as its DNS server, enabling internal name resolution across the VPN.
 - AllowedIPs = 0.0.0.0/0 established a full-tunnel VPN, routing all traffic through the WireGuard interface by default.
 - PersistentKeepalive = 25 ensured reliable connectivity for clients behind NAT devices and enterprise firewalls.
 - With IP forwarding and nftables configured on the server, clients on different VPN subnets (192.168.100.0/24, 192.168.150.0/24, and 192.168.200.0/24) were able to communicate through the centralized AWS VPN gateway.
-

Next Phase

Phase 3 will cover:

- Complete **WSTunnel over WireGuard** implementation (server and client)
- Running WSTunnel as a **systemd service**
- Modifying WireGuard to use 127.0.0.1:51820
- The **default routing loop problem** caused by AllowedIPs = 0.0.0.0/0
- Adding PostUp/PostDown static routes to preserve WSTunnel connectivity
- End-to-end verification and troubleshooting.

WireGuard VPN Configuration Guide

Phase 3 – WSTunnel Integration, Routing Fix & Service Configuration

1. Why WSTunnel?

Problem

The organization used a **FortiGate firewall** that blocked:

- UDP 51820 (WireGuard)
- ICMP
- Other non-approved outbound traffic

Result:

Client

|

UDP 51820

|

 Blocked

WireGuard could not establish a tunnel.

Solution

Encapsulate WireGuard UDP packets inside HTTPS.

WireGuard UDP

|

WSTunnel

|

HTTPS (TCP 443)

|

Internet

|

AWS EC2

|

WSTunnel

|

WireGuard

No changes were required to the VPN architecture itself—only the transport layer changed.

2. Final Architecture

Client

|

WireGuard

|

127.0.0.1:51820

|

WSTunnel Client

|

HTTPS TCP 443

|

Internet

|

AWS EC2 (443)

|

WSTunnel Server

|

127.0.0.1:51820

|

WireGuard Server

|

VPN Networks

3. Install WSTunnel

Download latest release

cd /tmp

wget

https://github.com/erebe/wstunnel/releases/download/v10.6.2/wstunnel_10.6.2_linux_amd64.tar.gz

tar -xzf wstunnel_10.6.2_linux_amd64.tar.gz

sudo mv wstunnel /usr/local/bin/

```
sudo chmod +x /usr/local/bin/wstunnel
```

Verify

```
wstunnel --version
```

Expected

```
wstunnel-cli 10.6.2
```

4. Configure Server

Create service

```
sudo nano /etc/systemd/system/wstunnel.service
```

Configuration

[Unit]

Description=WSTunnel Server

After=network-online.target

Wants=network-online.target

[Service]

Type=simple

ExecStart=/usr/local/bin/wstunnel server \

wss://0.0.0.0:443

Restart=always

RestartSec=5

User=root

[Install]

WantedBy=multi-user.target

Reload

```
sudo systemctl daemon-reload
```

Enable

```
sudo systemctl enable wstunnel
```

Start

```
sudo systemctl start wstunnel
```

Verify

```
sudo systemctl status wstunnel
```

5. Open HTTPS in nftables

Originally only WireGuard UDP was allowed.

Added rule

```
tcp dport 443 accept
```

Reload

```
sudo systemctl restart nftables
```

Verify

```
curl -vk https://13.205.137.92:443
```

Expected

HTTP 400 Invalid request

This is the expected result, because WSTunnel accepts only WebSocket upgrade requests, not normal HTTPS GET requests.

6. Configure Client

Modify WireGuard.

Original

Endpoint = 13.205.137.92:51820

New

Endpoint = 127.0.0.1:51820

WireGuard now communicates only with localhost.

7. Configure WSTunnel Client

Create service

```
sudo nano /etc/systemd/system/wstunnel.service
```

Contents

[Unit]

Description=WSTunnel Client

After=network-online.target

Wants=network-online.target

[Service]

Type=simple

ExecStart=/usr/local/bin/wstunnel client \

-L udp://51820:127.0.0.1:51820?timeout_sec=0 \

wss://13.205.137.92:443

Restart=always

RestartSec=5

User=root

[Install]

WantedBy=multi-user.target

Reload

sudo systemctl daemon-reload

Enable

sudo systemctl enable wstunnel

Start

sudo systemctl start wstunnel

8. Verify WSTunnel

Server

sudo systemctl status wstunnel

Client

sudo systemctl status wstunnel

Logs

```
sudo journalctl -u wstunnel -f
```

Socket

```
sudo ss -ltnp | grep 51820
```

Expected

127.0.0.1:51820

9. Major Problem Encountered

WireGuard configuration contained

AllowedIPs = 0.0.0.0/0

Meaning

Every packet was routed into WireGuard.

Unfortunately this also included WSTunnel.

Flow became

WSTunnel

↓

WireGuard

↓

WSTunnel

↓

WireGuard

↓

Infinite Loop

Symptoms

- No handshake
- Continuous retransmissions
- Increasing TX
- RX remained zero
- WSTunnel TCP connection timed out

10. Investigation

When WireGuard was stopped

```
sudo wg-quick down wg0
```

Testing

```
nc -vz 13.205.137.92 443
```

Succeeded.

After WireGuard started

```
sudo wg-quick up wg0
```

The same command timed out.

This confirmed that the VPN routing itself was intercepting the WSTunnel connection.

11. Root Cause

With

AllowedIPs = 0.0.0.0/0

WireGuard installs a default route.

Therefore

Destination

13.205.137.92

↓

wg0

↓

WSTunnel

↓

Attempts HTTPS

↓

Again through wg0

This recursive routing prevented the HTTPS tunnel from ever reaching the AWS server.

12. Solution — Static Host Route

A host-specific route was added so traffic to the AWS public IP always bypassed the VPN.

Manual command

```
sudo ip route add 13.205.137.92/32 \  
via 192.168.74.254 \  
dev ens33
```

Verify

```
ip route
```

Expected

```
13.205.137.92 via 192.168.74.254 dev ens33
```

This ensured that only traffic destined for the AWS public IP used the physical network interface, while all other traffic continued to use the VPN.

13. Permanent Route

To avoid manually adding the route after every reboot, it was integrated into the WireGuard configuration.

```
sudo nano /etc/wireguard/wg0.conf
```

Append

```
PostUp = ip route add 13.205.137.92/32 via 192.168.74.254 dev ens33
```

PostDown = ip route del 13.205.137.92/32 via 192.168.74.254 dev ens33

Now the exception route is created automatically when wg0 starts and removed when it stops.

14. Restart Services

```
sudo systemctl restart wstunnel
```

```
sudo systemctl restart wg-quick@wg0
```

or

```
sudo wg-quick down wg0
```

```
sudo wg-quick up wg0
```

15. Validation

Check WireGuard

```
sudo wg show
```

Expected

latest handshake

transfer received increasing

transfer sent increasing

Check route

```
ip route
```

Expected

default ...

13.205.137.92 via 192.168.74.254 dev ens33

192.168.100.0/24 dev wg0

Verify connectivity

ping 192.168.100.1

Verify WSTunnel

sudo journalctl -u wstunnel

Successful logs show accepted WebSocket tunnel connections rather than timeout errors.

16. Important Lessons Learned

- WSTunnel changes only the **transport** (UDP → HTTPS); the WireGuard VPN architecture, addressing, and peer configuration remain unchanged.
 - A response of **HTTP 400 Invalid request** from curl https://13.205.137.92:443 confirms the WSTunnel service is reachable; curl is not performing the required WebSocket upgrade.
 - The use of AllowedIPs = 0.0.0.0/0 introduced a routing loop because the WSTunnel client attempted to reach the AWS endpoint through the VPN tunnel it was responsible for establishing.
 - The routing issue was resolved by adding a host-specific static route for the AWS public IP through the physical gateway and automating it with PostUp and PostDown directives in wg0.conf.
 - After applying the routing fix, WireGuard established successful handshakes over WSTunnel, allowing VPN communication through restrictive firewalls using HTTPS (TCP 443).
-

Next Phase (Final)

Phase 4 will cover:

- Windows Server 2022 WireGuard client configuration
- Installing and configuring WSTunnel on Windows using NSSM
- Microsoft Visual C++ Runtime dependency
- Windows service creation
- Common troubleshooting and validation
- Final architecture and best practices.

WireGuard VPN Configuration Guide

Phase 4 – Windows Server Client Configuration, Validation & Troubleshooting

1. Objective

Integrate a Windows Server 2022 machine into the existing WireGuard VPN using the same AWS VPN server and WSTunnel-over-HTTPS transport.

Final communication path:

Windows Server

|

WireGuard

|

127.0.0.1:51820

|

WSTunnel Client

|

HTTPS (TCP 443)

|

AWS EC2

|

WSTunnel Server

|

WireGuard Server

|

VPN Network

2. Install WireGuard

Download and install the official **WireGuard for Windows** client.

After installation:

- Launch WireGuard.
- Create a new empty tunnel.
- Generate a key pair.

Copy the generated **public key** and add it to the AWS server.

3. Add Windows Peer on Server

Edit the server configuration.

```
sudo nano /etc/wireguard/wg0.conf
```

Example:

[Peer]

PublicKey = WINDOWS_PUBLIC_KEY

AllowedIPs = 192.168.150.40/32

Restart WireGuard.

```
sudo systemctl restart wg-quick@wg0
```

4. Configure Windows WireGuard Tunnel

Example configuration:

[Interface]

PrivateKey = WINDOWS_PRIVATE_KEY

Address = 192.168.150.40/24

DNS = 192.168.150.40

[Peer]

PublicKey = SERVER_PUBLIC_KEY

Endpoint = 127.0.0.1:51820

AllowedIPs = 0.0.0.0/0

PersistentKeepalive = 25

Important: After integrating WSTunnel, the endpoint is **127.0.0.1:51820**, not the AWS public IP.

5. Install WSTunnel

Download the Windows AMD64 release of WSTunnel and extract it, for example:

C:\wstunnel\

Files:

C:\wstunnel\wstunnel.exe

6. Test WSTunnel Manually

Open **Command Prompt (Administrator)** and run:

C:\wstunnel\wstunnel.exe client ^

-L udp://51820:127.0.0.1:51820?timeout_sec=0 ^

wss://13.205.137.92:443

If it starts without errors, continue with service creation.

7. Create Windows Service Using NSSM

Download **NSSM (Non-Sucking Service Manager)** and extract it.

Open an elevated Command Prompt.

nssm install WSTunnel

Configure:

Field	Value
Application	C:\wstunnel\wstunnel.exe
Startup Directory	C:\wstunnel
Arguments	client -L udp://51820:127.0.0.1:51820?timeout_sec=0 wss://13.205.137.92:443

Install the service.

Start it:

net start WSTunnel

8. Microsoft Visual C++ Runtime Issue

Problem

Initially, WSTunnel could not start on Windows.

Symptoms:

- Process exited immediately.
- NSSM reported service failures.

- No WSTunnel logs were produced.

Root Cause

The required **Microsoft Visual C++ Redistributable (2015–2022 x64)** was not installed.

Resolution

1. Install **VC_redist.x64.exe** (Microsoft Visual C++ Redistributable 2015–2022 x64).
2. Reboot the Windows Server.
3. Restart the WSTunnel service.

After installing the runtime, WSTunnel launched successfully.

9. Windows Firewall

Allow:

- WireGuard
- WSTunnel
- Loopback communication (127.0.0.1)
- Outbound HTTPS (TCP 443)

No inbound UDP 51820 exposure was required because WireGuard communicated locally with WSTunnel.

10. Validation

Check WSTunnel Service

sc query WSTunnel

or

services.msc

Status should be:

RUNNING

Check WireGuard

Open the WireGuard GUI.

Expected:

- Tunnel active
- Latest handshake updating

- RX/TX counters increasing
-

Connectivity

Ping the VPN gateway:

ping 192.168.100.1

Ping another VPN client:

ping 192.168.150.20

Test DNS:

nslookup <internal-hostname>

11. Common Issues Encountered

1. WSTunnel failed immediately

Cause

Missing Microsoft Visual C++ Runtime.

Resolution

Install the Visual C++ Redistributable and restart the system.

2. NSSM service would not remain running

Cause

WSTunnel executable terminated because its runtime dependency was missing.

Resolution

Install the Visual C++ Runtime, then recreate or restart the service.

3. No WireGuard handshake

Possible causes

- Incorrect server or client keys
 - Peer not added on the server
 - WSTunnel service not running
 - Incorrect endpoint (should be 127.0.0.1:51820 after WSTunnel integration)
 - HTTPS (TCP 443) blocked
-

4. WSTunnel cannot connect

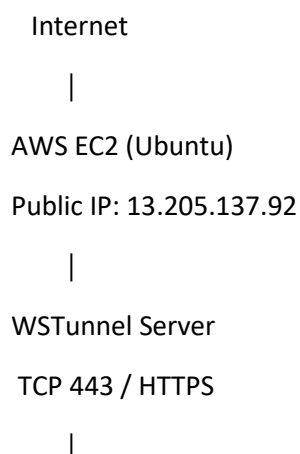
Check:

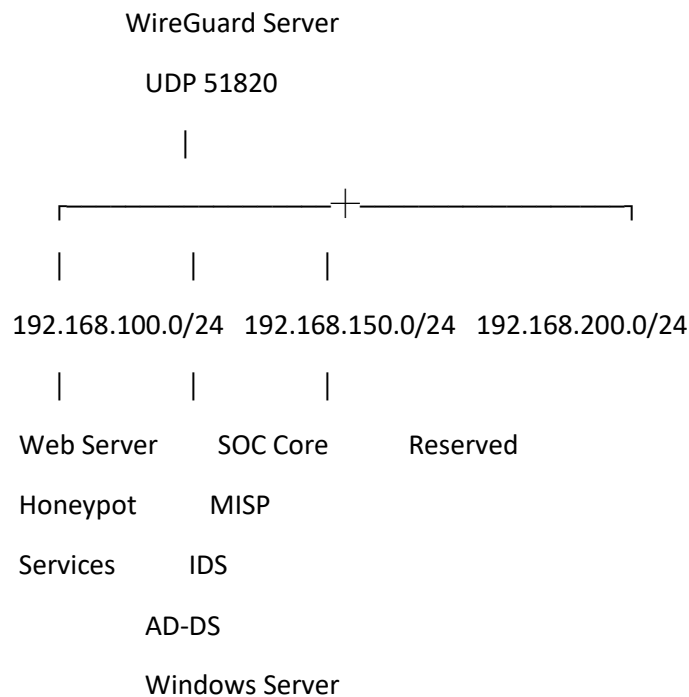
- AWS security group allows TCP 443.
- wstunnel.service is running on the server.
- nc -vz 13.205.137.92 443 succeeds from the client.
- The static route for the AWS public IP is present if using a Linux full-tunnel client.

12. Final Validation Checklist

Item	Verification
WireGuard installed	✓
WSTunnel installed	✓
WSTunnel service running	✓
Microsoft VC++ Runtime installed	✓
Windows peer added to server	✓
Handshake visible	✓
RX/TX counters increasing	✓
Ping to VPN gateway successful	✓
Cross-subnet communication successful	✓
DNS resolution through AD-DS working	✓

13. Final Architecture





14. Key Findings

- Native WireGuard was successfully extended with **WSTunnel**, allowing VPN connectivity through restrictive enterprise firewalls that permitted only HTTPS (TCP 443).
- Linux and Windows clients shared the same WireGuard architecture; only the transport layer changed, with WireGuard communicating locally with WSTunnel on 127.0.0.1:51820.
- On Windows Server, the primary deployment issue was the missing **Microsoft Visual C++ Redistributable (2015–2022 x64)**, which prevented wstunnel.exe from starting. Installing the runtime resolved the issue.
- Running WSTunnel as a persistent service (systemd on Linux and NSSM on Windows) ensured automatic startup and reliable VPN connectivity after reboots.
- After deployment, Linux and Windows systems successfully joined the same WireGuard VPN, used the centralized AD-DS DNS server, communicated across multiple VPN subnets, and maintained secure connectivity to the AWS-hosted infrastructure through HTTPS despite enterprise firewall restrictions.